data transformation with R and dplyr

prof. dr. nicolas meseth

1 why transform
2 the verbs
3 the pipe

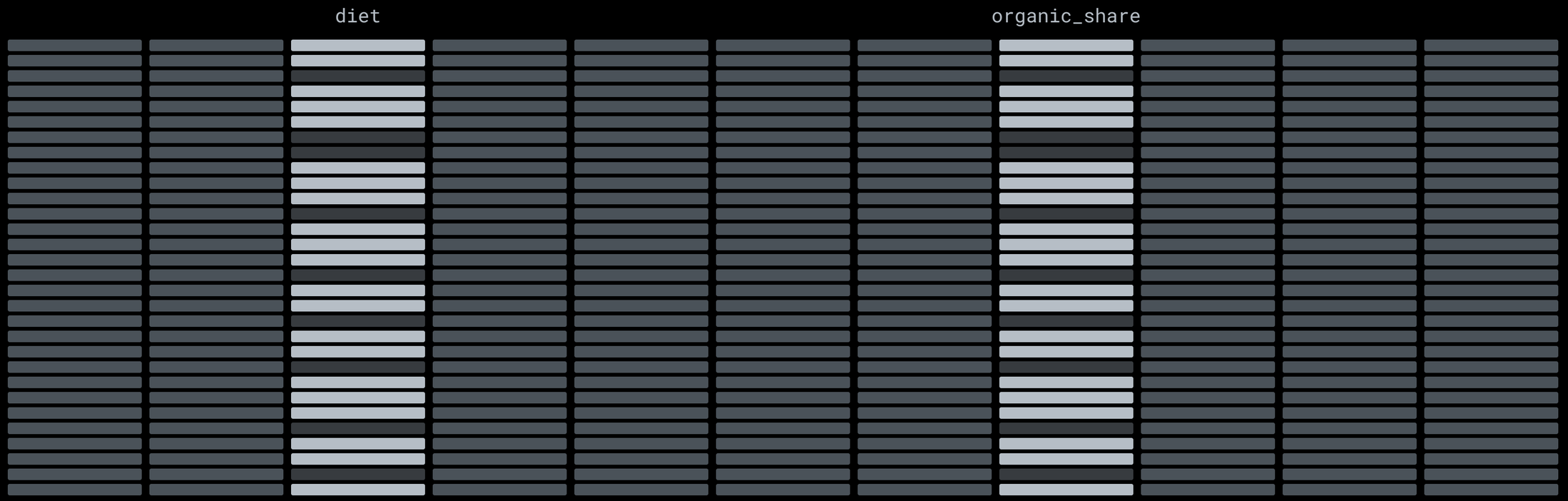


image: ai-generated (gpt-image-2.5-flare)

part 1

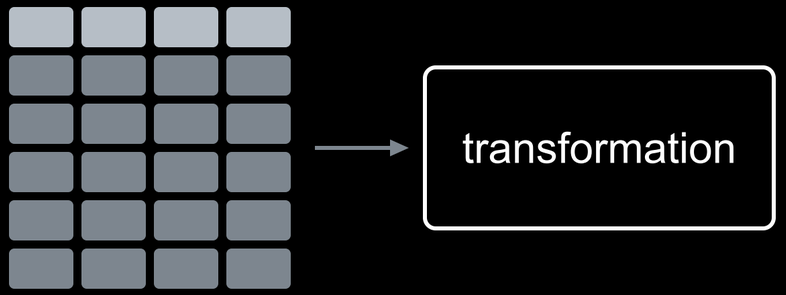
why transform

do vegetarians buy more organic food than others?



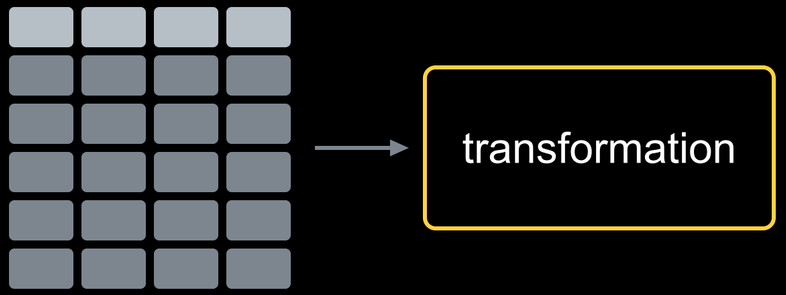
the answer is in the table, but you cannot read it off.

a table in, a table out



every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



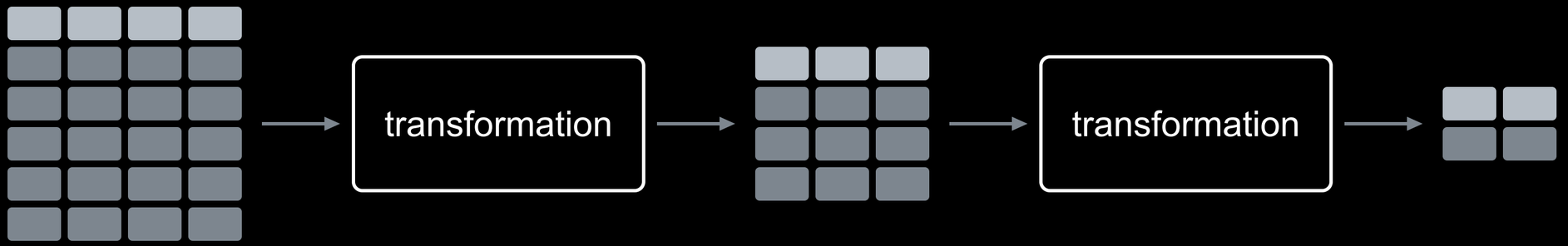
every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



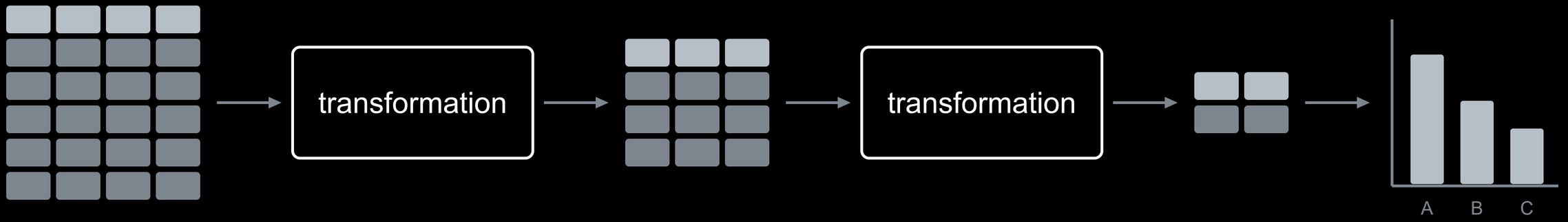
every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



every transformation takes a table and returns a table, which is why they can be chained.

six basic operations

select columns
which columns do I need?

filter rows
who is this about?

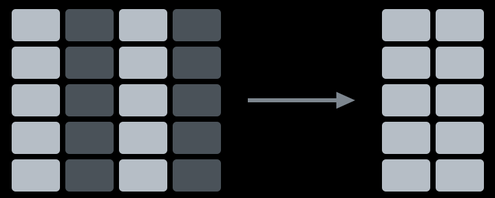
sort
what comes first?

compute new columns
what can be derived?

summarize
what is typical, how many?

group
how do subgroups differ?

six basic operations



select columns

which columns do I need?

filter rows

who is this about?

sort

what comes first?

compute new columns

what can be derived?

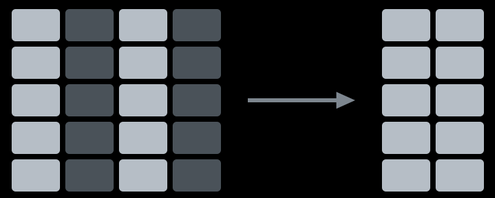
summarize

what is typical, how many?

group

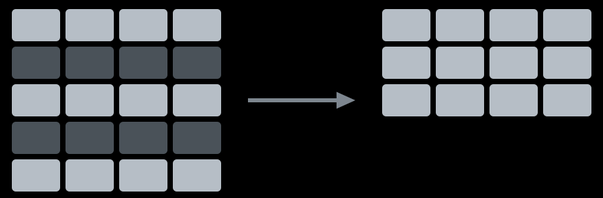
how do subgroups differ?

six basic operations



select columns

which columns do I need?



filter rows

who is this about?

sort

what comes first?

compute new columns

what can be derived?

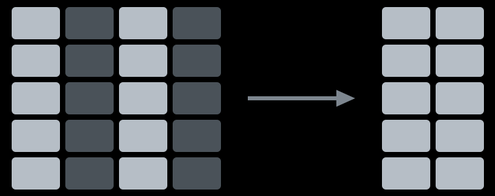
summarize

what is typical, how many?

group

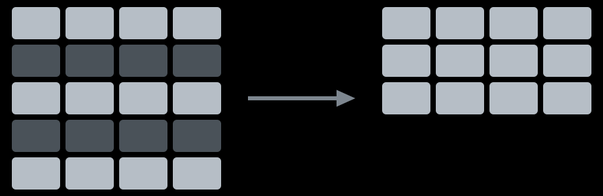
how do subgroups differ?

six basic operations



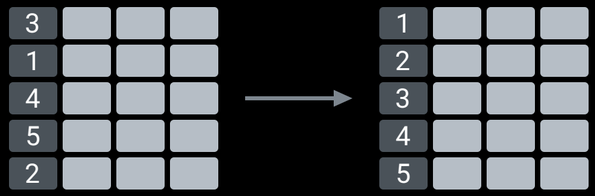
select columns

which columns do I need?



filter rows

who is this about?



sort

what comes first?

compute new columns

what can be derived?

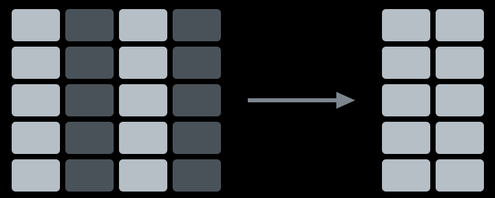
summarize

what is typical, how many?

group

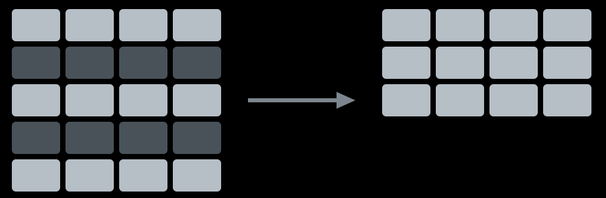
how do subgroups differ?

six basic operations



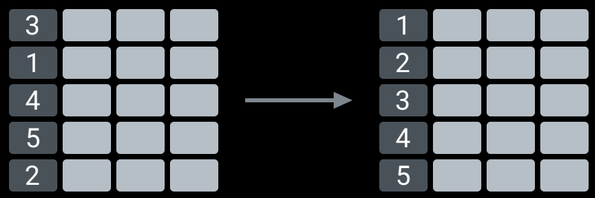
select columns

which columns do I need?



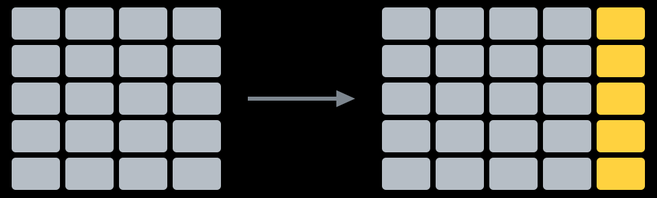
filter rows

who is this about?



sort

what comes first?



compute new columns

what can be derived?

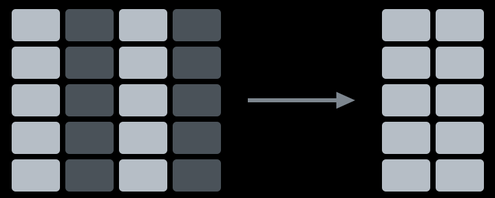
summarize

what is typical, how many?

group

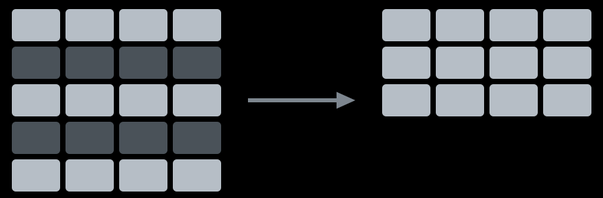
how do subgroups differ?

six basic operations



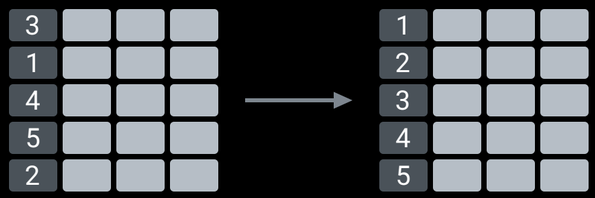
select columns

which columns do I need?



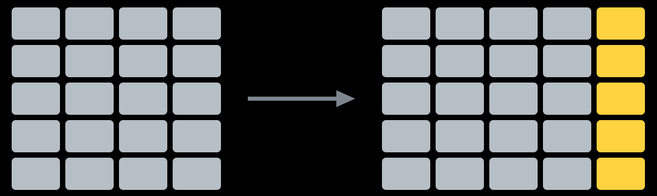
filter rows

who is this about?



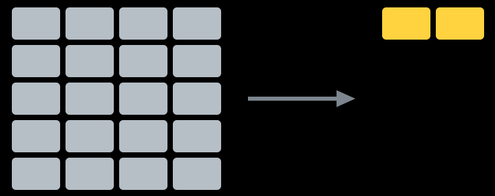
sort

what comes first?



compute new columns

what can be derived?



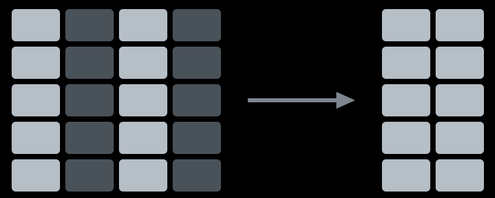
summarize

what is typical, how many?

group

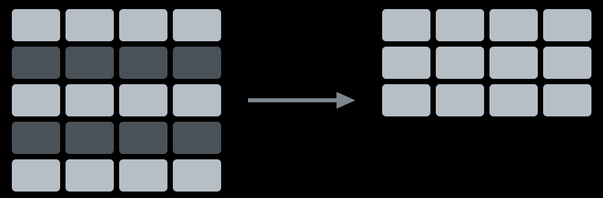
how do subgroups differ?

six basic operations



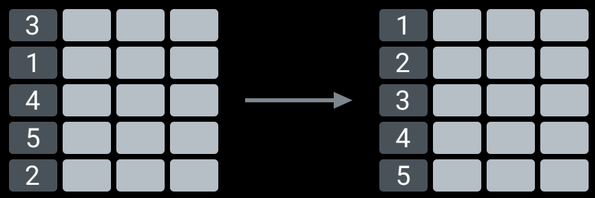
select columns

which columns do I need?



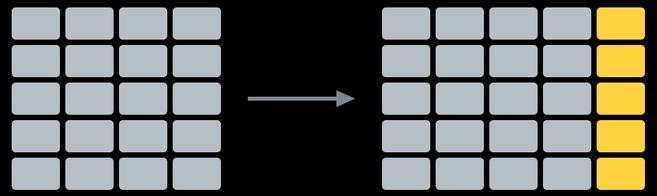
filter rows

who is this about?



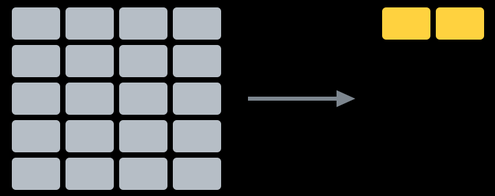
sort

what comes first?



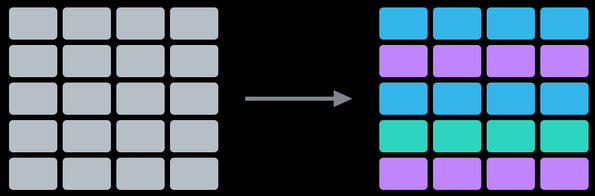
compute new columns

what can be derived?



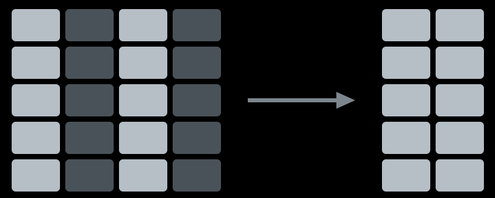
summarize

what is typical, how many?



group

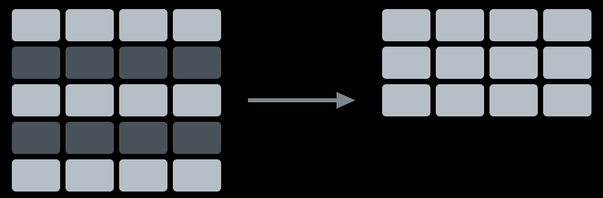
how do subgroups differ?



select columns

which columns do I need?

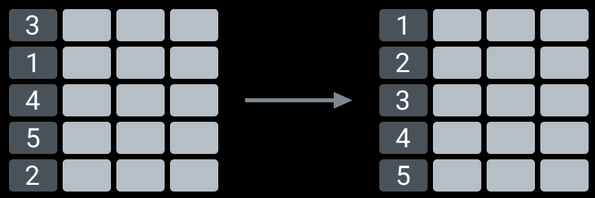
`select()`



filter rows

who is this about?

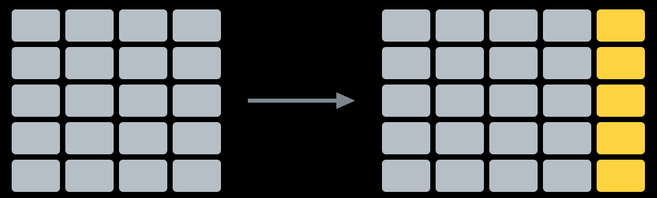
`filter()`



sort

what comes first?

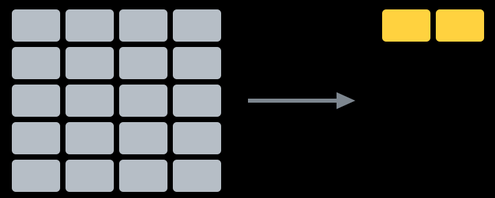
`arrange()`



compute new columns

what can be derived?

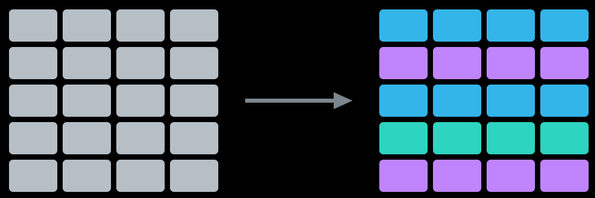
`mutate()`



summarize

what is typical, how many?

`summarize()`



group

how do subgroups differ?

`group_by()`

the same ideas everywhere

operation	dplyr	SQL	pandas	excel
select columns	`select()`	`SELECT`	`df[columns]`	hide columns
filter rows	`filter()`	`WHERE`	`df.query()`	filter
sort	`arrange()`	`ORDER BY`	`sort_values()`	sort
new columns	`mutate()`	`SELECT … AS`	`df.assign()`	formula column
summarize	`summarize()`	`COUNT, AVG`	`df.agg()`	pivot table
group	`group_by()`	`GROUP BY`	`df.groupby()`	pivot table

part 2

the verbs

dplyr names its functions after what they do

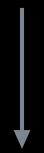
```
library(tidyverse)
```

```
filter(survey, age >= 50)
```

dplyr names its functions after what they do

```
library(tidyverse)
```

the first argument is a table

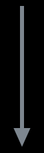


```
filter(survey, age >= 50)
```

dplyr names its functions after what they do

```
library(tidyverse)
```

the first argument is a table



```
filter(survey, age >= 50)
```



columns by their bare name, without quotes and without $

dplyr names its functions after what they do

```
library(tidyverse)
```

the first argument is a table

```
filter(survey, age >= 50)
```



a new table comes out,
survey stays as it was

columns by their bare name, without quotes and without $

dplyr names its functions after what they do

```
library(tidyverse)
```

```
survey |>
  filter(age >= 50)
```

we always write it like this; more on the pipe |> later

six rows of the survey

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

organic_share: share of organic food when shopping, from 1 (hardly any) to 5 (almost all)

```
survey |>
  select(id, diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  select(id, diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  select(id, diet)
```

id	diet
1	omnivore
2	omnivore
3	omnivore
4	omnivore
5	flexitarian
6	vegetarian

different ways to say which columns

```
survey |>
  select(id, diet)
```

by name

```
survey |>
  select(id:diet)
```

a range, from id to diet

```
survey |>
  select(starts_with("organic"))
```

by a pattern in the name

```
survey |>
  select(id, starts_with("organic"))
```

combined: by name and by pattern

```
survey |>
  select(2:5)
```

by position, the 2nd to the 5th column

```
survey |>
  select(-organic_share)
```

all but this one

```
survey |>
  select(where(is.numeric))
```

by the type of the column

more helpers, such as `ends_with()` and `contains()`, in the help: `?select`

```
survey |>
  filter(age >= 50)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  filter(age >= 50)
```

id	age	diet	organic_share	
1	56	omnivore	2	✓
2	55	omnivore	2	✓
3	47	omnivore	1	✗
4	32	omnivore	1	✗
5	74	flexitarian	2	✓
6	48	vegetarian	2	✗

```
survey |>
  filter(age >= 50)
```

id	age	diet	organic_share	
1	56	omnivore	2	
2	55	omnivore	2	
5	74	flexitarian	2	

```
survey |>
  filter(diet == "vegan")

survey |>
  filter(age >= 50 & diet != "omnivore")

survey |>
  filter(diet %in% c("vegan", "vegetarian"))

survey |>
  filter(!is.na(organic_share))
```

⚠ = assigns, == compares

`==`	equal
`!=`	not equal
`&`	and
`|`	or
`%in%`	one of
`is.na()`	missing

comparisons and logical operators

```
survey |>
  arrange(age)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  arrange(age)
```

id	↑ age	diet	organic_share
4	32	omnivore	1
3	47	omnivore	1
6	48	vegetarian	2
2	55	omnivore	2
1	56	omnivore	2
5	74	flexitarian	2

```
survey |>
  arrange(desc(age))
```

id	↓ age	diet	organic_share
5	74	flexitarian	2
1	56	omnivore	2
2	55	omnivore	2
6	48	vegetarian	2
3	47	omnivore	1
4	32	omnivore	1

```
survey |>
  arrange(age)
```

ascending, from the youngest

```
survey |>
  arrange(desc(age))
```

descending, from the oldest

```
survey |>
  arrange(-age)
```

also descending, a shortcut for numbers

```
survey |>
  arrange(diet, desc(age))
```

by diet, within it from the oldest

```
survey |>
  mutate(decade = age %/% 10 * 10)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  mutate(decade = age %/% 10 * 10)
```

id	age	diet	organic_share	decade
1	56	omnivore	2	
2	55	omnivore	2	
3	47	omnivore	1	
4	32	omnivore	1	
5	74	flexitarian	2	
6	48	vegetarian	2	

```
survey |>
  mutate(decade = age %/% 10 * 10)
```

id	age	diet	organic_share	decade
1	56	omnivore	2	50
2	55	omnivore	2	50
3	47	omnivore	1	40
4	32	omnivore	1	30
5	74	flexitarian	2	70
6	48	vegetarian	2	40

one row in, one row out

```
survey |>
  mutate(
    decade = age %/% 10 * 10,
    organic_fan = organic_share >= 4
  )
```

one row in, one row out

```
survey |>
  mutate(
    decade = age %/% 10 * 10,
    organic_fan = organic_share >= 4
  )
```

id	age	organic_share	decade	organic_fan
1	56	2	50	FALSE
2	55	2	50	FALSE
3	47	1	40	FALSE
4	32	1	30	FALSE
5	74	2	70	FALSE
6	48	2	40	FALSE
7	36	5	30	TRUE
8	26	3	20	FALSE

```
survey |>
  summarize(n = n(), mean_organic = mean(organic_share))
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  summarize(n = n(), mean_organic = mean(organic_share))
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

n
6

```
survey |>
  summarize(n = n(), mean_organic = mean(organic_share))
```

n	mean_organic
6	1.67

frequently used summary functions

```
survey |>
  summarize(
    n = n(),
    mean_organic = mean(organic_share, na.rm = TRUE),
    oldest = max(age)
  )
```

`n()`	number of rows
`mean()`	average
`median()`	middle value
`min()`	smallest
`max()`	largest
`sd()`	spread

a selection, there are many more

one missing value makes the mean NA, unless `na.rm = TRUE`

```
survey |>
  group_by(diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  group_by(diet)
```

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  group_by(diet)
```

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  group_by(diet) |>
  summarize(n = n(), mean_age = mean(age))
```

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
survey |>
  group_by(diet) |>
  summarize(n = n(), mean_age = mean(age))
```

diet	n	mean_age
flexitarian	1	74
omnivore	4	47.5
vegetarian	1	48

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()



live.hsos.uber.space/q

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()

live.hsos.uber.space/q

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()



live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() |> summarize()

mutate()



live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() |> summarize()

mutate()

live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() |> summarize()

mutate()

live.hsos.uber.space/q

part 3

the pipe

nested calls are read from the inside out

```
arrange(
  summarize(
    group_by(
      filter(survey, age >= 40),
      diet
    ),
    n = n()
  ),
  desc(n)
)
```

nested calls are read from the inside out

```
arrange(
  summarize(
    group_by(
      filter(survey, age >= 40),
      diet
    ),
    n = n()
  ),
  desc(n)
)
```

nested calls are read from the inside out

```
   arrange(
     summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

nested calls are read from the inside out

```
   arrange(
3    summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

nested calls are read from the inside out

```
4  arrange(
3    summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

```
older <- filter(survey, age >= 40)
grouped <- group_by(older, diet)
counts <- summarize(grouped, n = n())
result <- arrange(counts, desc(n))
```

readable, but full of names nobody needs again.

the pipe passes the table on

you write

```
x |> f(y)
```

the pipe passes the table on

you write

```
x |> f(y)
```

R runs

```
f(x, y)
```

the pipe passes the table on

you write

```
x |> f(y)
```

R runs

```
f(x, y)
```

read |> as "and then"

%>% in older code does the same

the pipe passes the table on

you write

```
x |> f(y)
```

R runs

```
f(x, y)
```

read |> as "and then"

%>% in older code does the same

`ctrl` + `shift` + `m` types |> in RStudio and Positron, cmd instead of ctrl on a Mac

a pipeline



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

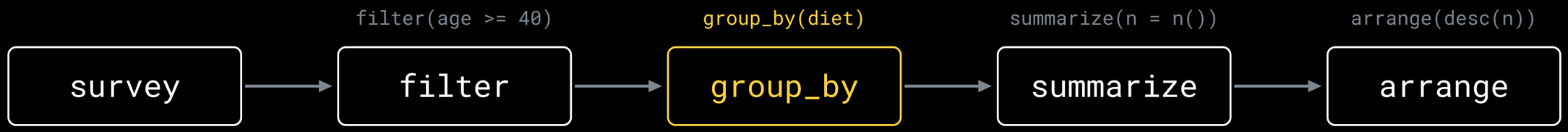
a pipeline



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

a pipeline



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

Groups: diet [3]

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

a pipeline



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

Groups: diet [3]

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

diet from a to z

diet	n
flexitarian	1
omnivore	3
vegetarian	1

a pipeline



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

Groups: diet [3]

age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

diet from a to z

diet	n
flexitarian	1
omnivore	3
vegetarian	1

diet	n
omnivore	3
flexitarian	1
vegetarian	1

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

the same, as you would write it

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

and then count,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

and then count,

and then sort by the count.

take the survey

keep the rows with an organic share

split by diet

average and count per group

sort by the average

take the survey

keep the rows with an organic share

split by diet

average and count per group

sort by the average



```
survey
```

take the survey → `survey`

keep the rows with an organic share → `filter()`

split by diet

average and count per group

sort by the average

take the survey → `survey`

keep the rows with an organic share → `filter()`

split by diet → `group_by()`

average and count per group

sort by the average

take the survey	→	`survey`
keep the rows with an organic share	→	`filter()`
split by diet	→	`group_by()`
average and count per group	→	`summarize()`
sort by the average		

take the survey	→	`survey`
keep the rows with an organic share	→	`filter()`
split by diet	→	`group_by()`
average and count per group	→	`summarize()`
sort by the average	→	`arrange()`

take the survey	→	`survey`
keep the rows with an organic share	→	`filter()`
split by diet	→	`group_by()`
average and count per group	→	`summarize()`
sort by the average	→	`arrange()`

the real task is breaking the question into steps, because translating each step into a verb is the easy part.

```
survey |>
  filter(!is.na(organic_share)) |>
  group_by(diet) |>
  summarize(
    n = n(),
    mean_organic = mean(organic_share)
  ) |>
  arrange(desc(mean_organic))
```

```
survey |>
  filter(!is.na(organic_share)) |>
  group_by(diet) |>
  summarize(
    n = n(),
    mean_organic = mean(organic_share)
  ) |>
  arrange(desc(mean_organic))
```

diet	n	mean_organic
vegan	36	3.69
vegetarian	134	3.03
pescetarian	36	2.86
flexitarian	742	2.86
NA	28	2.43
omnivore	1815	2.21

`select()`	columns
`filter()`	rows
`arrange()`	order
`mutate()`	new columns
`summarize()`	one row
`group_by()`	per group
`|>`	and then

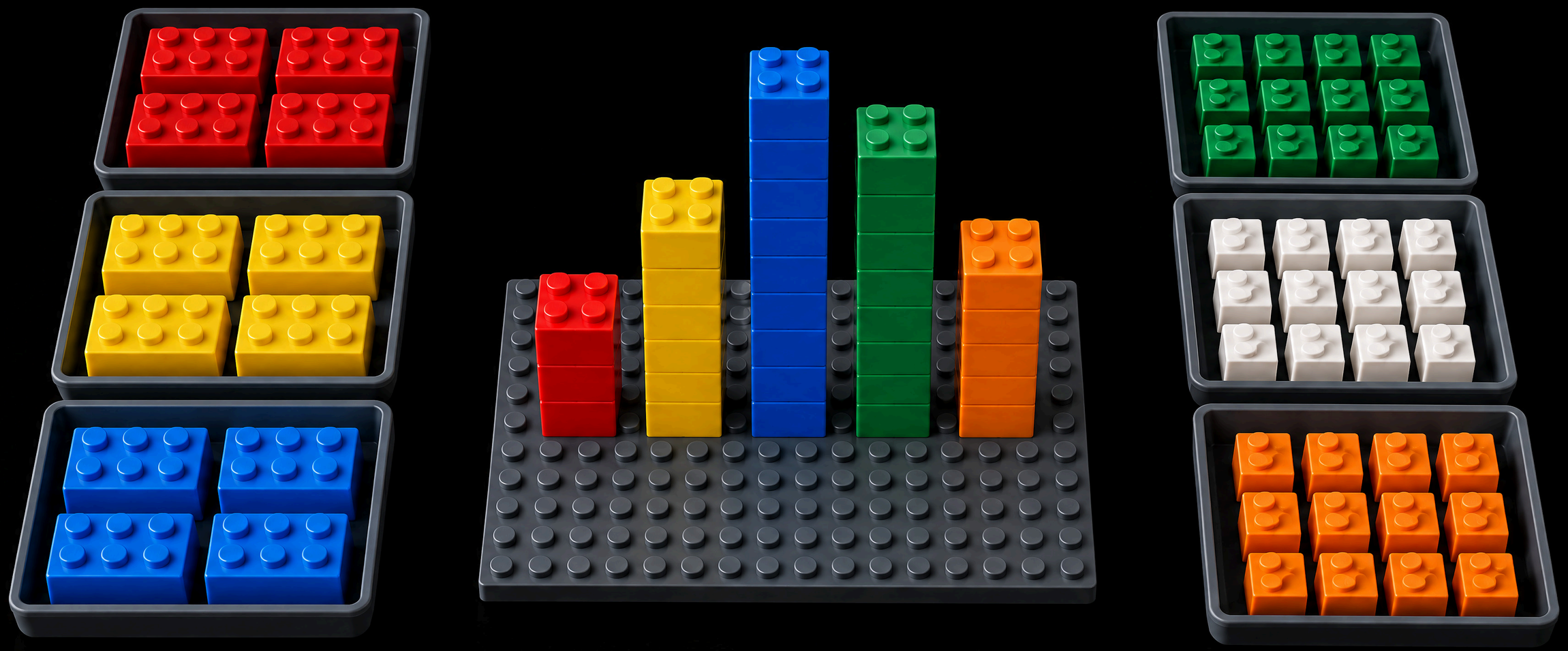


image: ai-generated (gpt-image-2.5-flare)